

Plotly入門

インタラクティブな可視化

rkskmt

1

静的から動的へ

2

Plotlyとは

- Plotlyは **インタラクティブな可視化** を作るライブラリ
 - 拡大・縮小、ホバーで値を確認
 - 凡例クリックで表示・非表示
 - 3Dグラフを回転
- matplotlibと同じく、Pythonから使える
- 図は **ブラウザで動くHTML** として出力される

3

今日やること

- 棒グラフ・散布図・折れ線・ヒストグラム・3D散布図を **plotly.express** で作る
- Pandasの集計結果をPlotlyに渡す
- 図をHTMLとして保存して共有する
- 最後に、**世界一有名な「動くグラフ」** を自分の手で再現する

i matplotlibとの「描き比べ」

- 散布図・折れ線・ヒストグラムは、matplotlibでも作った **同じ種類の図**
- 同じ図を別の道具で作りながら、**道具ごとの得意・不得意の違い** に注目

4

インストール & import

ターミナルからインストール：

```
Shell
$ pip install plotly
```

冒頭で import（可視化ツールを集めた **plotly.express** を **px** としてインポートするのが一般的）：

```
Code
1 import plotly.express as px
```



まずDataFrameを用意する

- PlotlyはPandasの **DataFrame** と相性がよい
- 「どの列をx軸に、どの列をy軸にするか」を **列名で指定するだけ**
- まずは以下をVS Codeにコピーして実行

Code

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "名前": ["田中", "佐藤", "鈴木", "高橋", "伊藤"],
5     "クラス": ["A", "B", "A", "C", "A"],
6     "点数": [82, 75, 91, 68, 57],
7 })
8
9 print(df)
```

Result

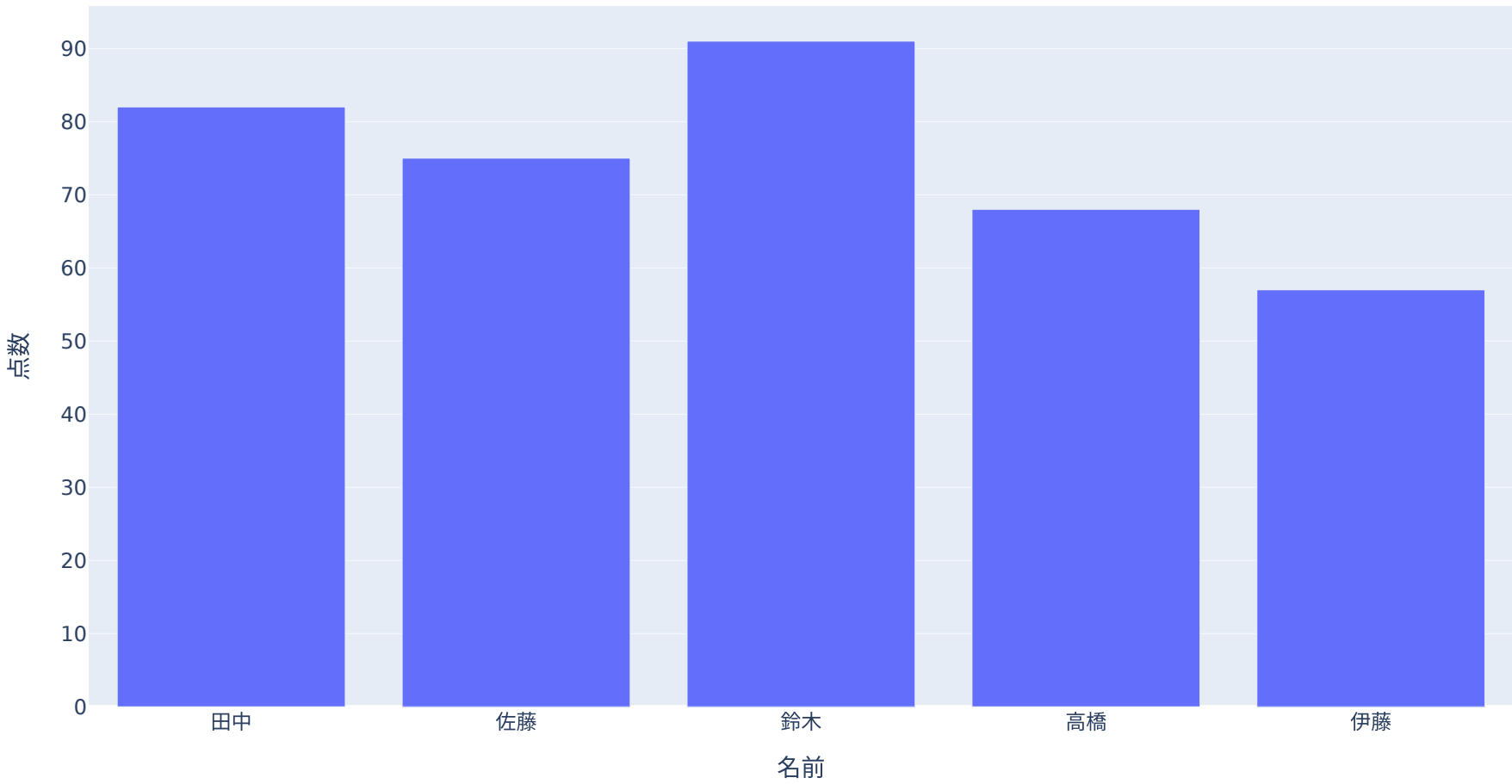
	名前	クラス	点数
0	田中	A	82
1	佐藤	B	75
2	鈴木	A	91
3	高橋	C	68
4	伊藤	A	57

棒グラフを作る

- `px.bar()` に **DataFrameと列名** を渡す
- `fig.show()` を実行すると、**ブラウザが開いて 図が表示される**

Code

```
1 import plotly.express as px
2
3 fig = px.bar(df, x="名前", y="点数")
4 fig.show()
```



- できた図は動く： **棒にマウスを乗せると値が出る**。右上のツールバーでズームや画像保存もできる

Plotly Expressの基本形

- `plotly.express` は、短く書くための高レベルAPI
- 基本形はどのグラフでも似ている

Code

```
1 fig = px.グラフの種類(
2     データフレーム,
3     x="x軸に使う列名",
4     y="y軸に使う列名",
5 )
```

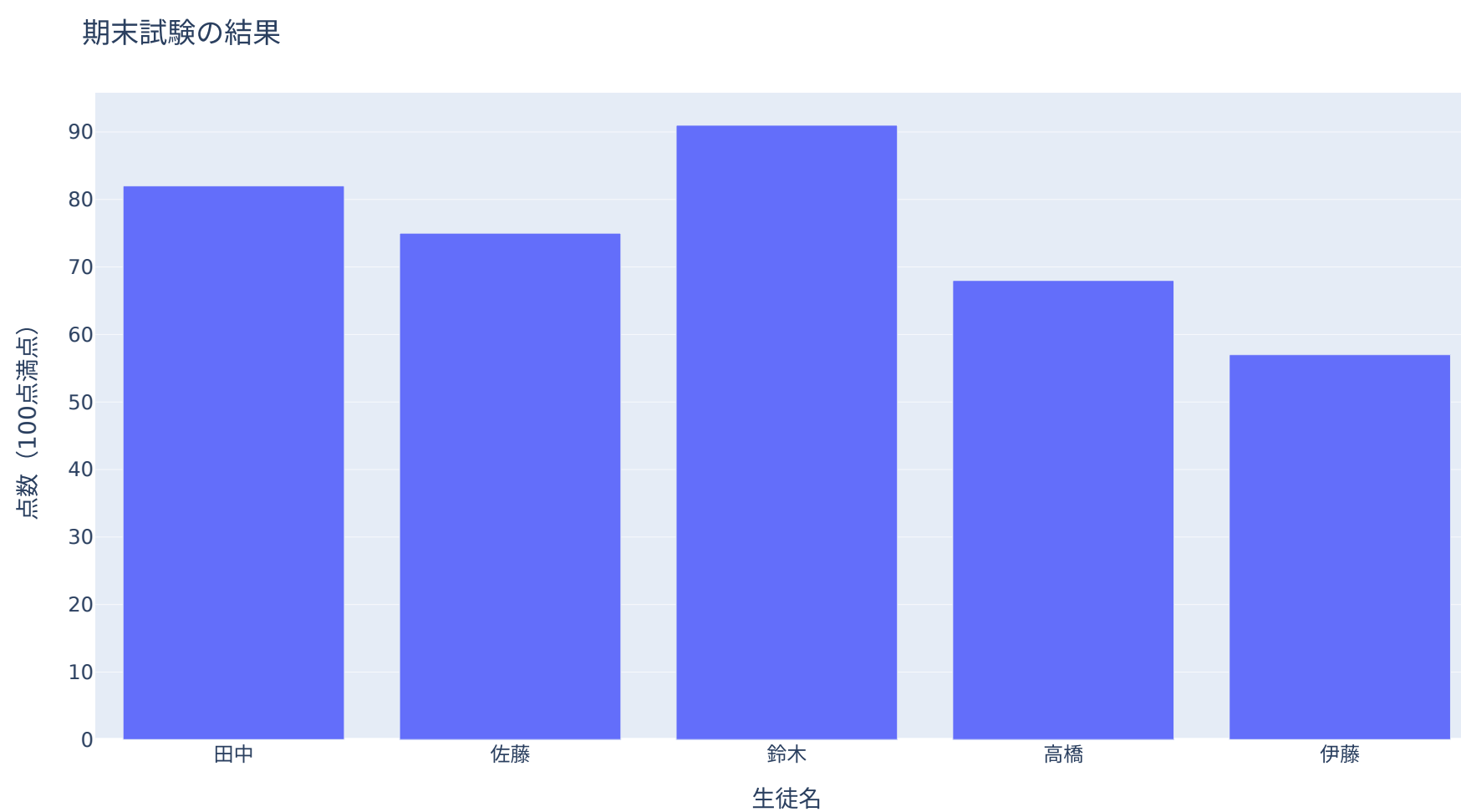
グラフの種類	関数
棒グラフ	<code>px.bar()</code>
散布図	<code>px.scatter()</code>
折れ線	<code>px.line()</code>
ヒストグラム	<code>px.histogram()</code>
3D散布図	<code>px.scatter_3d()</code>

図にラベルとタイトルをつける

- **title=** でタイトル
- **labels=** には **辞書** を渡す — **{列名: 表示したい軸ラベル}**
→ 列名はそのまま、**表示だけ** 変えられる

Code

```
1 fig = px.bar(  
2     df,  
3     x="名前",  
4     y="点数",  
5     title="期末試験の結果",  
6     labels={"名前": "生徒名", "点数": "点数 (100点満点)"},  
7 )  
8 fig.show()
```

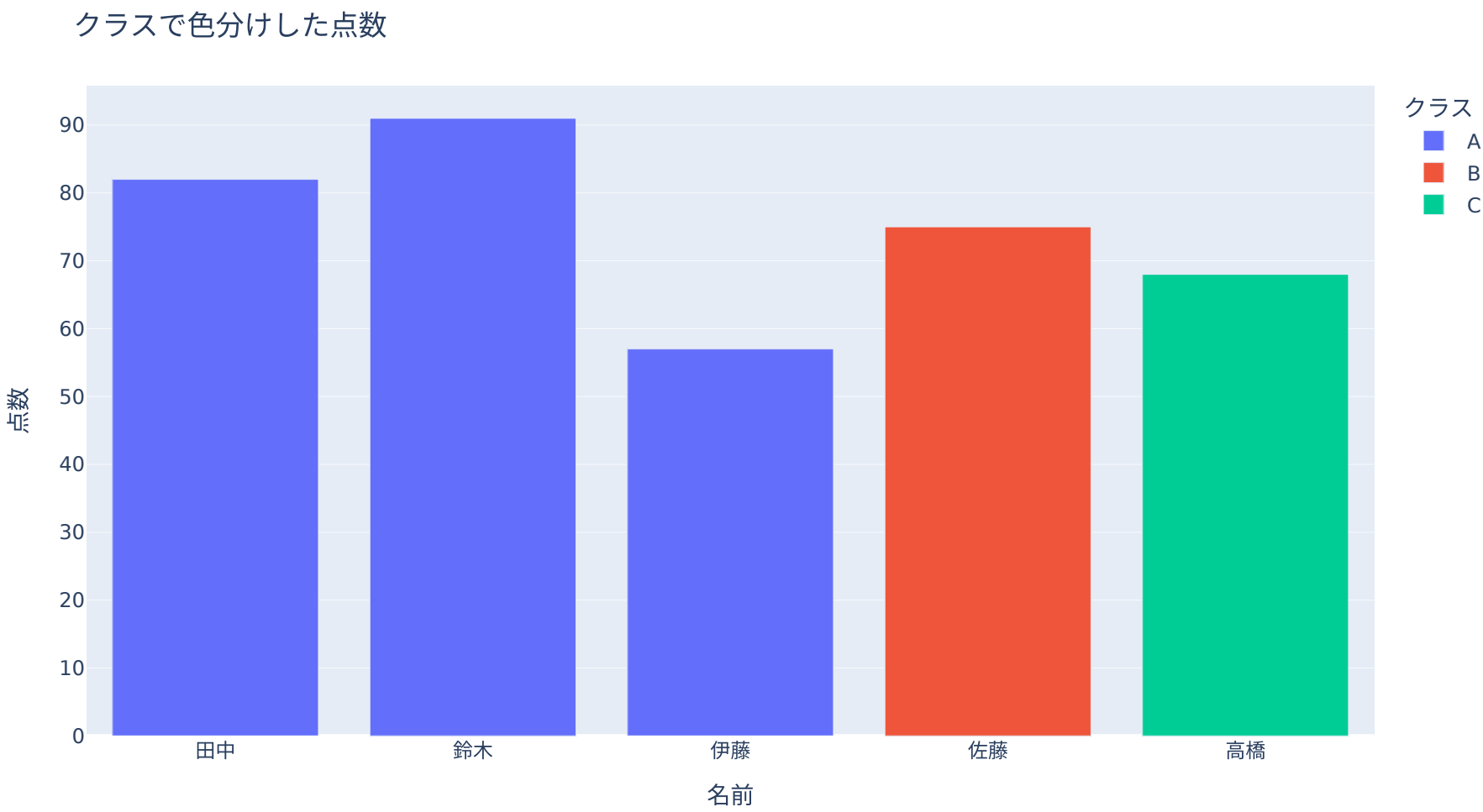


色分けする

- **color=** に列名を指定すると、その列の値で色分けできる
- カテゴリの違いを見せたいときに便利

Code

```
1 fig = px.bar(  
2     df,  
3     x="名前",  
4     y="点数",  
5     color="クラス",  
6     title="クラスで色分けした点数",  
7 )  
8 fig.show()
```



💡 何が便利？

- 凡例をクリックすると、特定のクラスだけ表示・非表示にできる
- 棒にマウスを乗せると、値が表示される

散布図

- **px.scatter()** で散布図を作る
- データは[Pandas入門](#)で使った **students.csv** (20人×8列)
→ CSVを **data** フォルダに置いて以下で読み込み

Code

```
1 import pandas as pd
2 import plotly.express as px
3
4 df_scores = pd.read_csv("data/students.csv")
5 print(df_scores.head())
```

Result

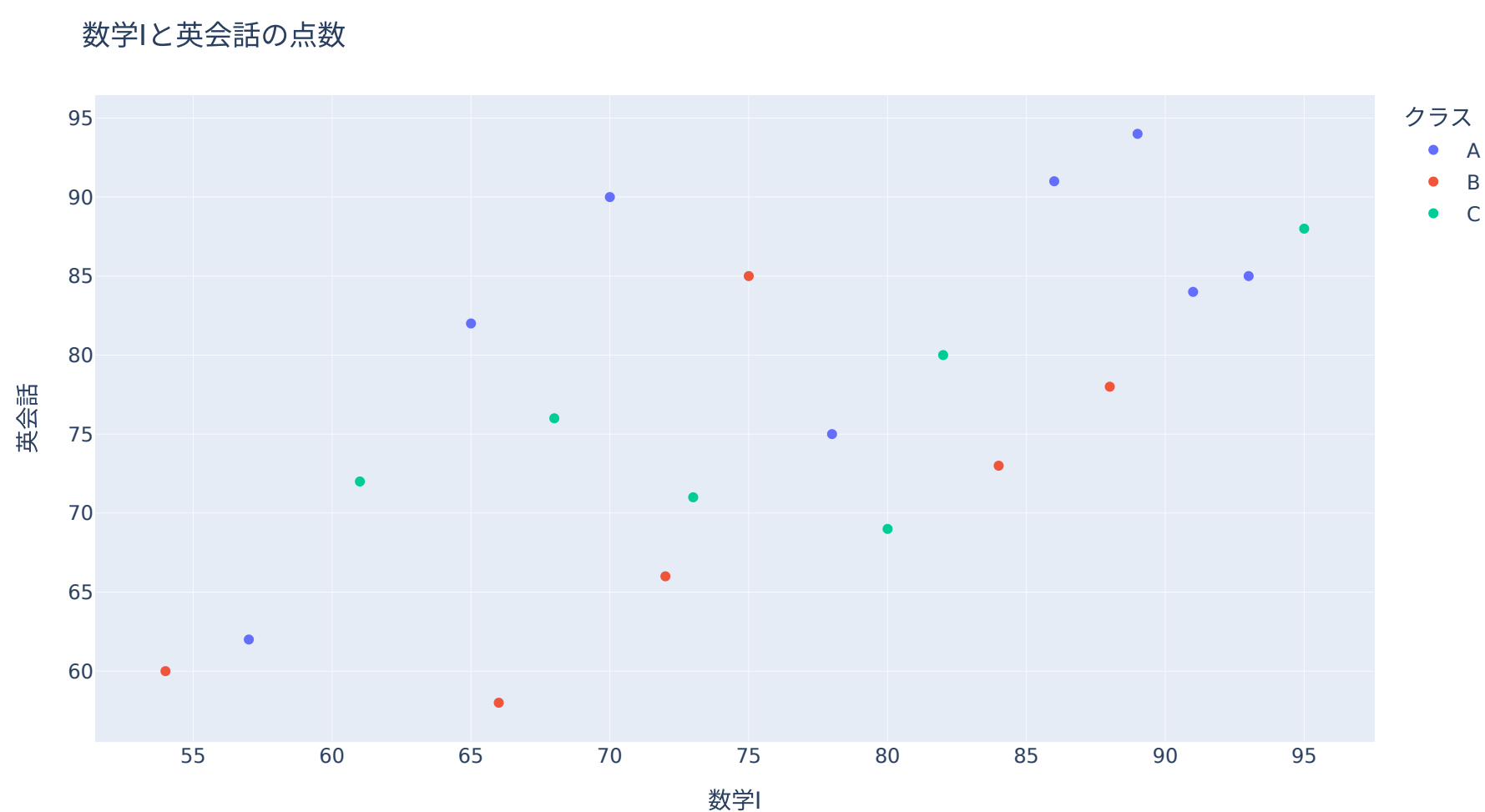
	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	A	1	80	70	90	85	2
1	佐藤	B	1	65	75	85	72	4
2	鈴木	A	2	88	91	84	90	1
3	高橋	C	2	72	68	76	80	3
4	伊藤	A	1	58	57	62	65	6

散布図のコード

- **x** と **y** に数値列を指定する
- **hover_name=** に指定した列が、ホバーしたときの **見出し** になる

Code

```
1 fig = px.scatter(  
2     df_scores,  
3     x="数学I",  
4     y="英会話",  
5     color="クラス",  
6     hover_name="名前",  
7     title="数学Iと英会話の点数",  
8 )  
9 fig.show()
```



💡 外れた点、誰？

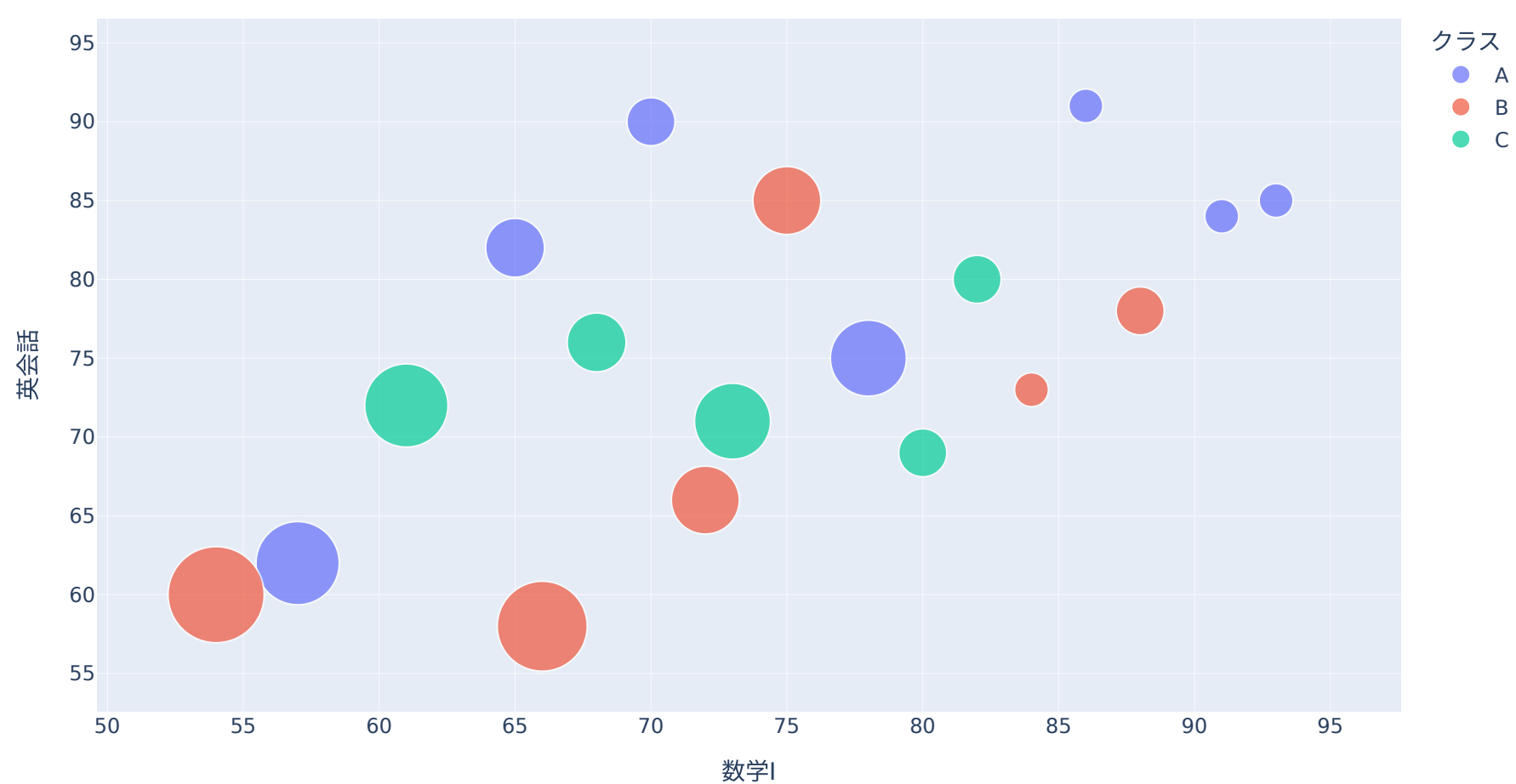
- 離れた点にマウスを乗せると **誰なのか** その場で分かる
- 静止画の散布図では、外れた点が誰なのかは分からなかった

点の大きさに3つ目の量を見せる

- **size=** に数値列を指定すると、点の大きさが **3つ目の量** になる（今回は欠席日数）
→ **size_max=** は一番大きい点のサイズ
- 欠席 **0日** の点は大きさ0で消えてしまう — できた **fig** は **fig.update_~()** 系のメソッドで微調整できるので、**sizemin** で最小の大きさを決めておく

Code

```
1 fig = px.scatter(  
2     df_scores,  
3     x="数学I",  
4     y="英会話",  
5     color="クラス",  
6     size="欠席日数",  
7     size_max=40,  
8     hover_name="名前",  
9 )  
10 fig.update_traces(marker=dict(sizemin=4)) # 欠席0日の点も見えるように  
11 fig.show()
```



- 大きい点ほど欠席が多い — 成績との関係が見えるか？

折れ線グラフ

- `px.line()` で折れ線グラフを作る
- 時間変化や順序のあるデータに向いている
- 題材は[データの可視化](#) でも見た **東京150年の年平均気温**（気象庁）
→ [tokyo_temp_annual.csv](#) を **data** フォルダに置く

Code

```
1 import pandas as pd
2 import plotly.express as px
3
4 df_temp = pd.read_csv("data/tokyo_temp_annual.csv")
5 print(df_temp.head())
```

Result

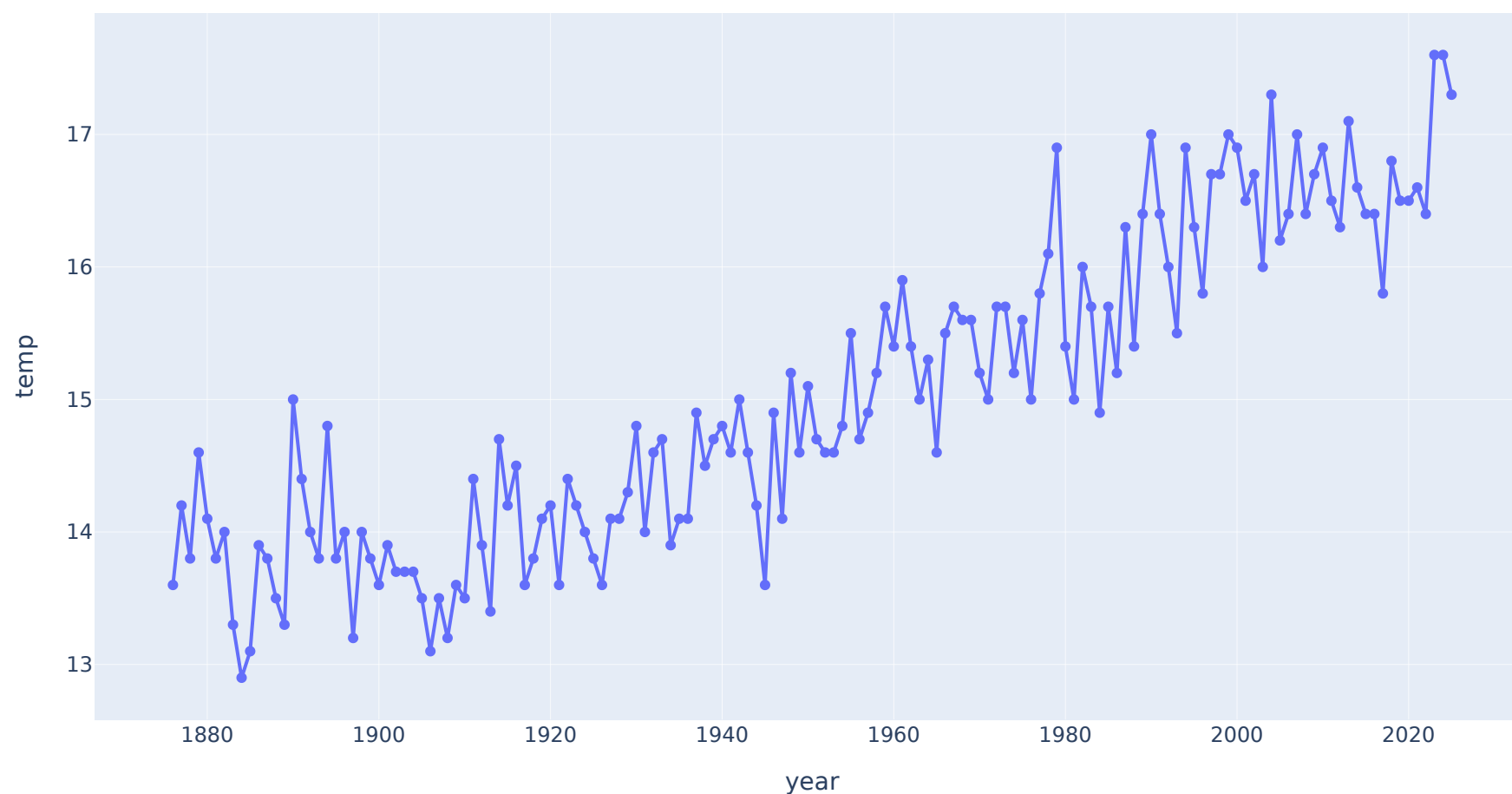
	year	temp
0	1876	13.6
1	1877	14.2
2	1878	13.8
3	1879	14.6
4	1880	14.1

折れ線グラフのコード

- **markers=True** で点＋線（matplotlibの **"o-"** と同じ役目）
- 軸ラベルには **列名がそのまま** 入る（変えたければ **labels=**）

Code

```
1 fig = px.line(df_temp, x="year", y="temp", markers=True)
2 fig.show()
```



💡 自分の指でチェリーピックしてみる

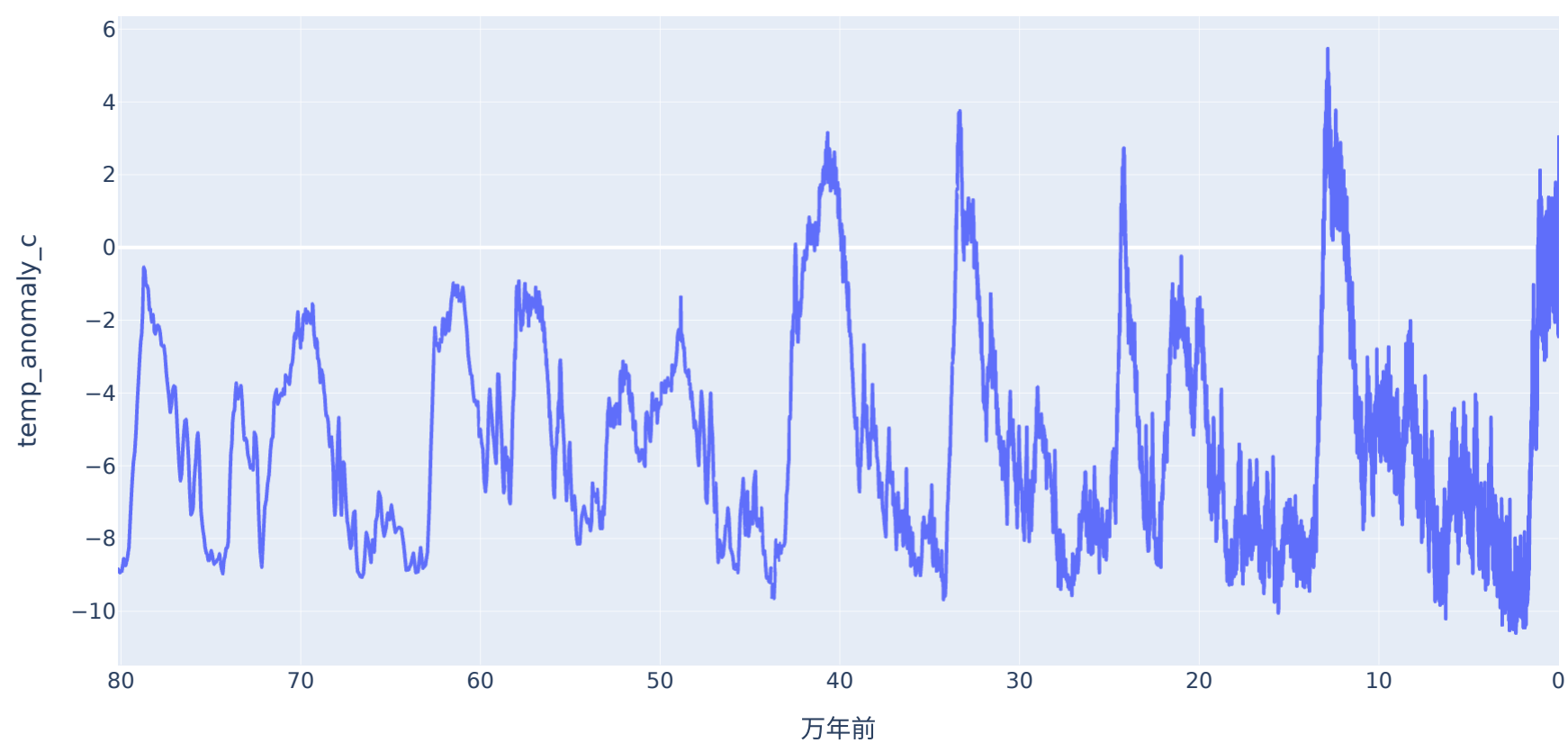
- **ドラッグで2006～2017年あたりを選択** してズーム → 「寒冷化」に見える
- **「期間のチェリーピック」** で見た **切り取り** が、指一本でできてしまう
- **ダブルクリックで全体に戻る** — 全体を確認するクセをつける

ズームアウト：80万年の気温

- 南極の氷から復元した気温（EPICA Dome C） — **80万年前** まで遡れる
- 縦軸は現在との気温差：**0°C=現在の水準**（直近1000年の平均）、マイナスほど寒い
- **epica_temp.csv** を **data** フォルダに置く

Code

```
1 df_ice = pd.read_csv("data/epica_temp.csv")
2 df_ice["万年前"] = df_ice["age_kyr_bp"] / 10 # 千年前 → 万年前
3
4 fig = px.line(df_ice, x="万年前", y="temp_anomaly_c")
5 fig.update_xaxes(autorange="reversed") # 昔を左、現在を右に
6 fig.show()
```



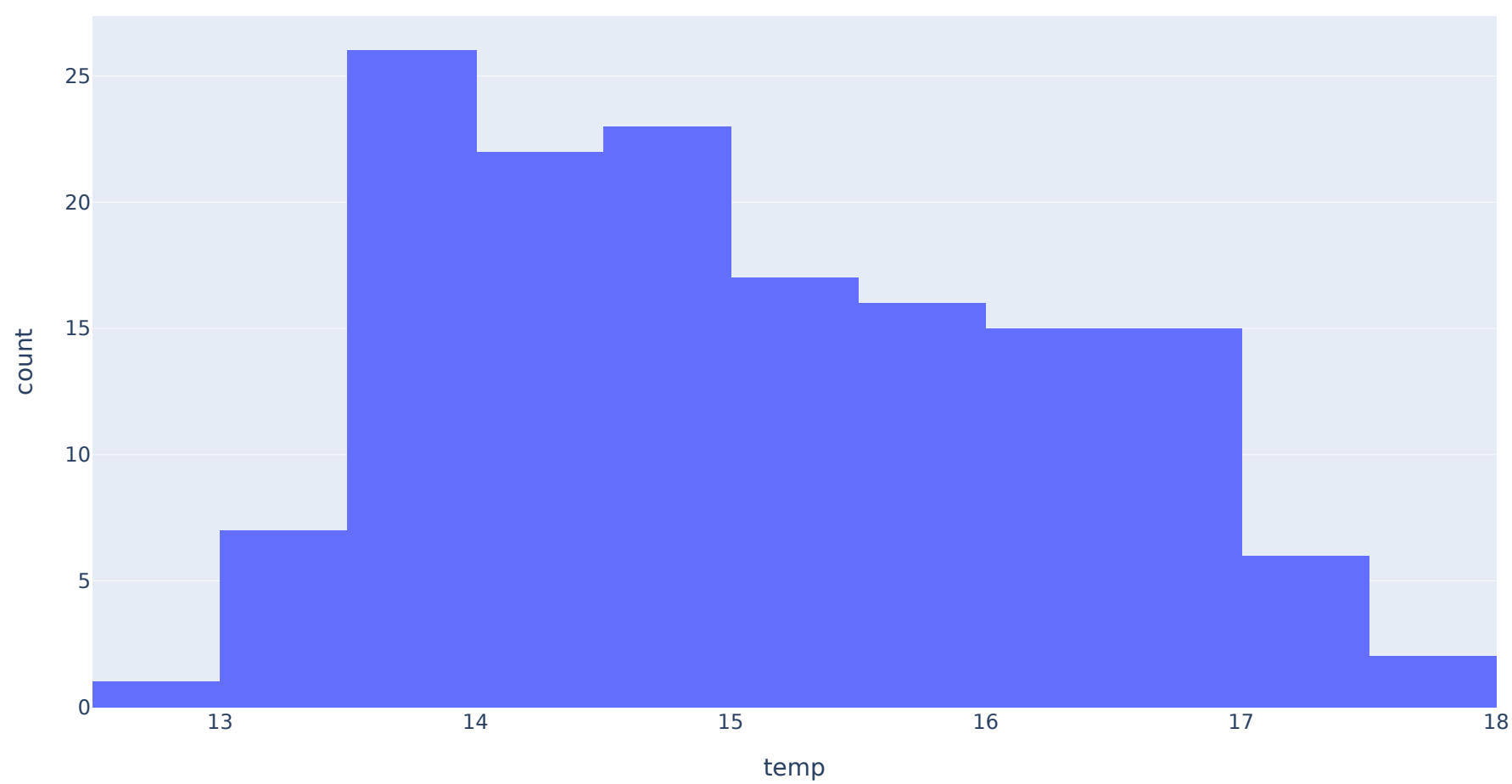
- ドラッグでズーム：氷期と間氷期のあいだを **10°C以上** も往復している
- 東京150年の変化は、この巨大なリズムの **右端のごく一部**
- 同じ「気温」でも、**見せるスパンで物語が変わる**

ヒストグラム

- `px.histogram()` で **分布** を見られる
- `nbins=` は区間（ビン）の数 — ビンの数で分布の見え方が化けるのは、matplotlibのときと同じ
- 同じ150年の気温データ — 折れ線は **変化**、ヒストグラムは **分布** を見せる

Code

```
1 fig = px.histogram(df_temp, x="temp", nbins=20)
2 fig.show()
```



- 棒にマウスを乗せると **どの気温帯が何年分か** まで分かる

18

Pandasの集計結果をPlotlyへ

19

クラスごとの平均点を計算

- **Pandas入門** の **groupby()** の復習 — **students.csv** でクラスごとの平均点を出す
- **groupby()** の結果は **Series** (**index** がクラス、**values** が平均点)

```
Code
1 import pandas as pd
2
3 df_students = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5
6 df_students["平均点"] = df_students[subjects].mean(axis=1)
7 avg = df_students.groupby("クラス")["平均点"].mean()
8 print(avg)
```

Result	
	クラス
A	80.062500
B	71.625000
C	77.583333
Name: 平均点, dtype: float64	

- ただしPlotly Expressの基本形は **DataFrame + 列名** — **Series** のままでは渡しにくい

SeriesをDataFrameに戻す

- **reset_index()** で、**index** を普通の列に戻した **DataFrame** が得られる

```
Code
1 import pandas as pd
2 import plotly.express as px
3
4 df_students = pd.read_csv("data/students.csv")
5 subjects = ["現代文", "数学I", "英会話", "物理"]
6 df_students["平均点"] = df_students[subjects].mean(axis=1)
7 avg = df_students.groupby("クラス")["平均点"].mean()
8
9 avg_df = avg.reset_index()
10 print(avg_df)
```

Result		
	クラス	平均点
0	A	80.062500
1	B	71.625000
2	C	77.583333

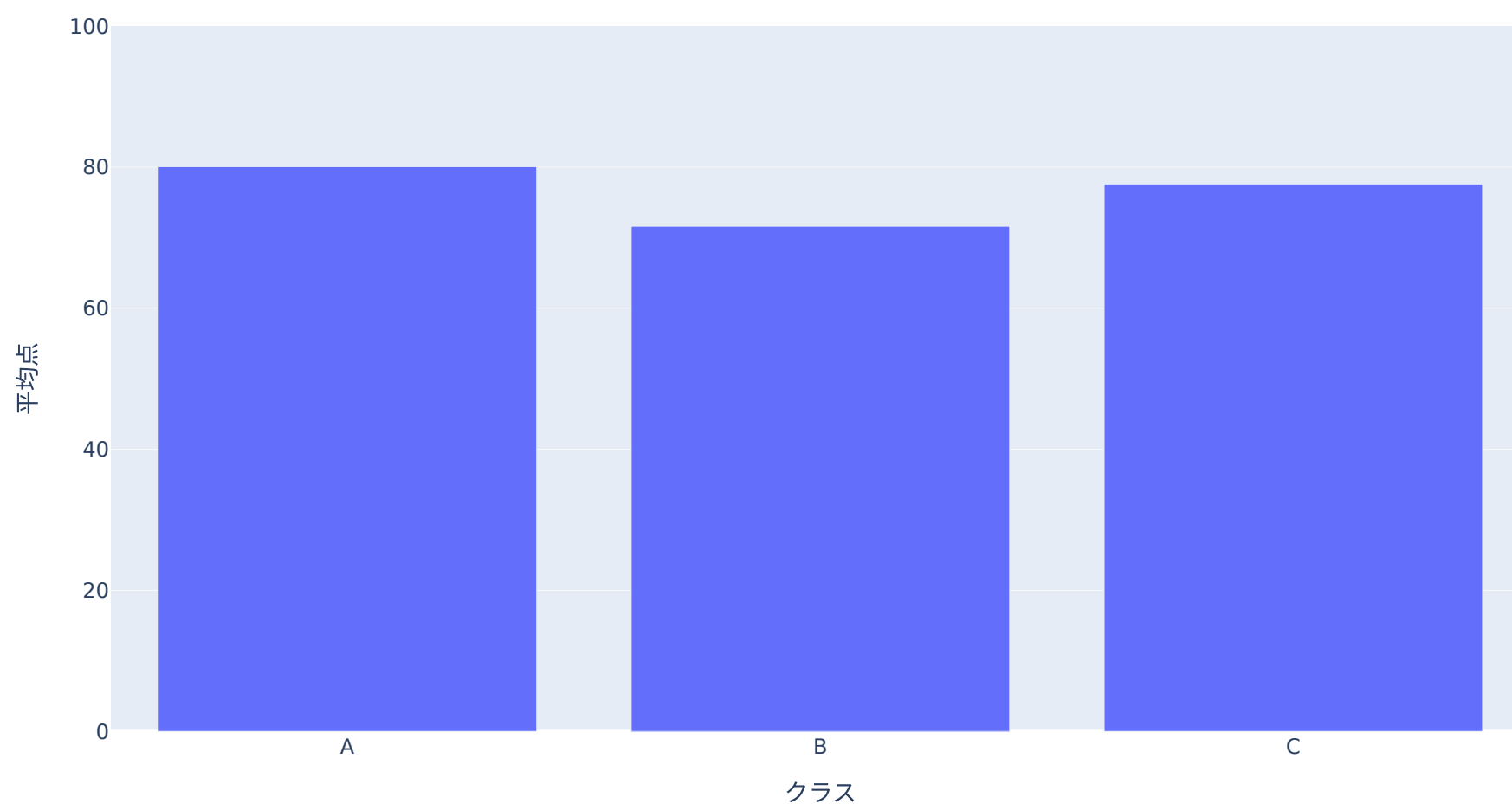
- **クラス** と **平均点** の **2列のDataFrame** になった — これで **x=**、**y=** に列名で渡せる

集計結果をPlotlyで可視化

- y軸は `update_yaxes(range=[最小, 最大])` で固定できる — 平均点なら **0点～満点** で正直に見せる
- matplotlibでは `plt.ylim(0, 100)` と書いた、そのPlotly版

Code

```
1 fig = px.bar(avg_df, x="クラス", y="平均点")
2 fig.update_yaxes(range=[0, 100]) # 棒グラフは0から見せる
3 fig.show()
```



22

そのままDataFrameで受け取る

- `groupby(..., as_index=False)` とすると、最初から **DataFrame** で結果を受け取れる
- Plotlyに渡す前提なら、`reset_index()` が要らないこちらが1行短い

Code

```
1 import pandas as pd
2
3 df_students = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5 df_students["平均点"] = df_students[subjects].mean(axis=1)
6
7 avg_df = df_students.groupby("クラス", as_index=False)["平均点"].mean()
8 print(avg_df)
```

Result

	クラス	平均点
0	A	80.062500
1	B	71.625000
2	C	77.583333

23

演習：クラスごとの平均点をPlotlyで描く



Pandas入門でmatplotlibを使って描いた「クラスごとの平均点」の棒グラフを、Plotlyで描き直せ。

0. **`students.csv`** [🔗](#) をダウンロードして **`data`** フォルダに置く（配置済みならそのまま）
1. **`pd.read_csv()`** で読み込み、4科目の平均点の列を追加
2. **`groupby("クラス", as_index=False)`** でクラスごとの平均点を集計
3. **`px.bar()`** で棒グラフを描く（**`color="クラス"`** で色分け、y軸は **0～100** に固定）

▶ 解答例を見る

24

3Dと書き出し

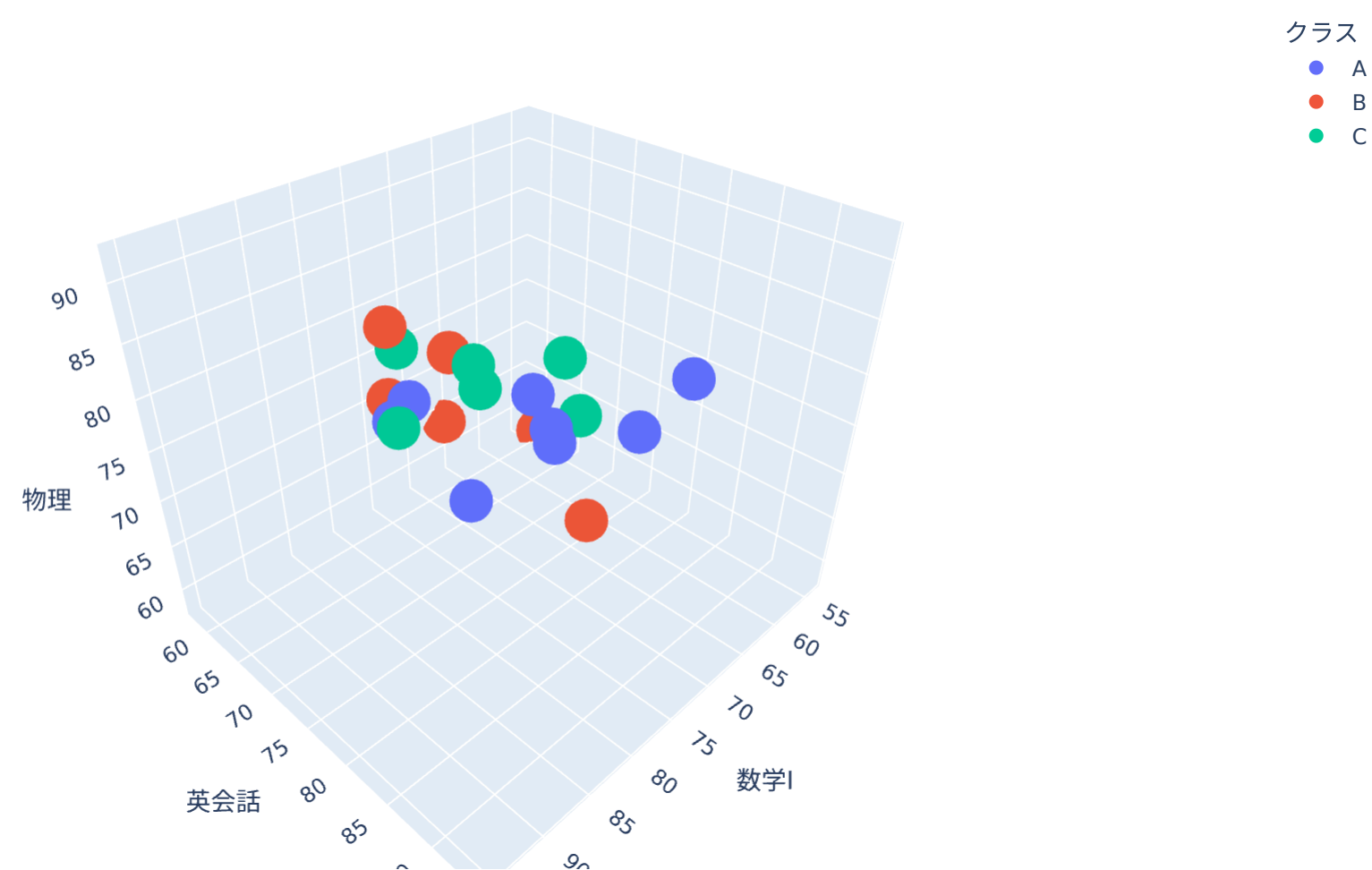
25

3D散布図

- `px.scatter_3d()` に `x`・`y`・`z` の **3つの数値列** を渡す
- **ドラッグで回転**、ホバーで名前 — 3科目（数学I・英会話・物理）の関係を探索できる

Code

```
1 import pandas as pd
2 import plotly.express as px
3
4 df_3d = pd.read_csv("data/students.csv")
5
6 fig = px.scatter_3d(
7     df_3d,
8     x="数学I",
9     y="英会話",
10    z="物理",
11    color="クラス",
12    hover_name="名前",
13 )
14 fig.show()
```



① 3Dは「回せる道具」で

- matplotlibでも3D図は作れるが、**視点を動かさない3Dは読みにくい**
- 回転・ズーム・ホバーで探索できるPlotlyは、3D散布図と相性がよい

HTMLとして保存する

- Plotlyの図は `write_html()` でHTMLとして保存できる
- ブラウザさえあれば誰でも開ける — **動くまま** 共有できる

Code

```
1 fig.write_html("graph.html")
```

- 何も指定しないと **Plotly本体ごと** HTMLに埋め込まれる（約4MB）
→ ファイルは重いが、**ネットがなくても動く**
- ここまで触ってきた動くグラフも、こうして保存したHTMLの埋め込み

i

include_plotlyjs で本体の置き場所を選べる

指定	HTMLのサイズ	閲覧時のネット
(指定なし)	重い（約4MB）	不要
"cdn"	軽い	必要
"directory"	軽い（ plotly.min.js を 同じフォルダに書き出す）	不要

ネットのない場所でも動かしたい資料は、指定なし か **"directory"** を選ぶ

matplotlibとPlotlyの使い分け

目的	向いている道具
レポートに貼る静止画	matplotlib
論文・印刷用の細かい調整	matplotlib
ホバーで値を見たい	Plotly
拡大して細部を見たい	Plotly
Webページで共有したい	Plotly
3Dを回して見たい	Plotly

- どちらが上ではなく、目的で使い分ける
- まずはmatplotlibで基本、Webで動かしたいときにPlotly

世界一有名な「動くグラフ」

Gapminder “Wealth & Health of Nations”

- データの可視化の実例集 で見た 有名な「動くグラフ」
 - 横軸：1人あたりGDP、縦軸：平均寿命、バブルの大きさ：人口
- Plotlyには **このデータが同梱** されている — ダウンロード不要

Code

```
1 import plotly.express as px
2
3 gap = px.data.gapminder()
4 print(gap.head())
5 print(gap.shape)
```

Result

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	\
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	
	iso_num							
0	4							
1	4							
2	4							
3	4							
4	4							
(1704, 8)								

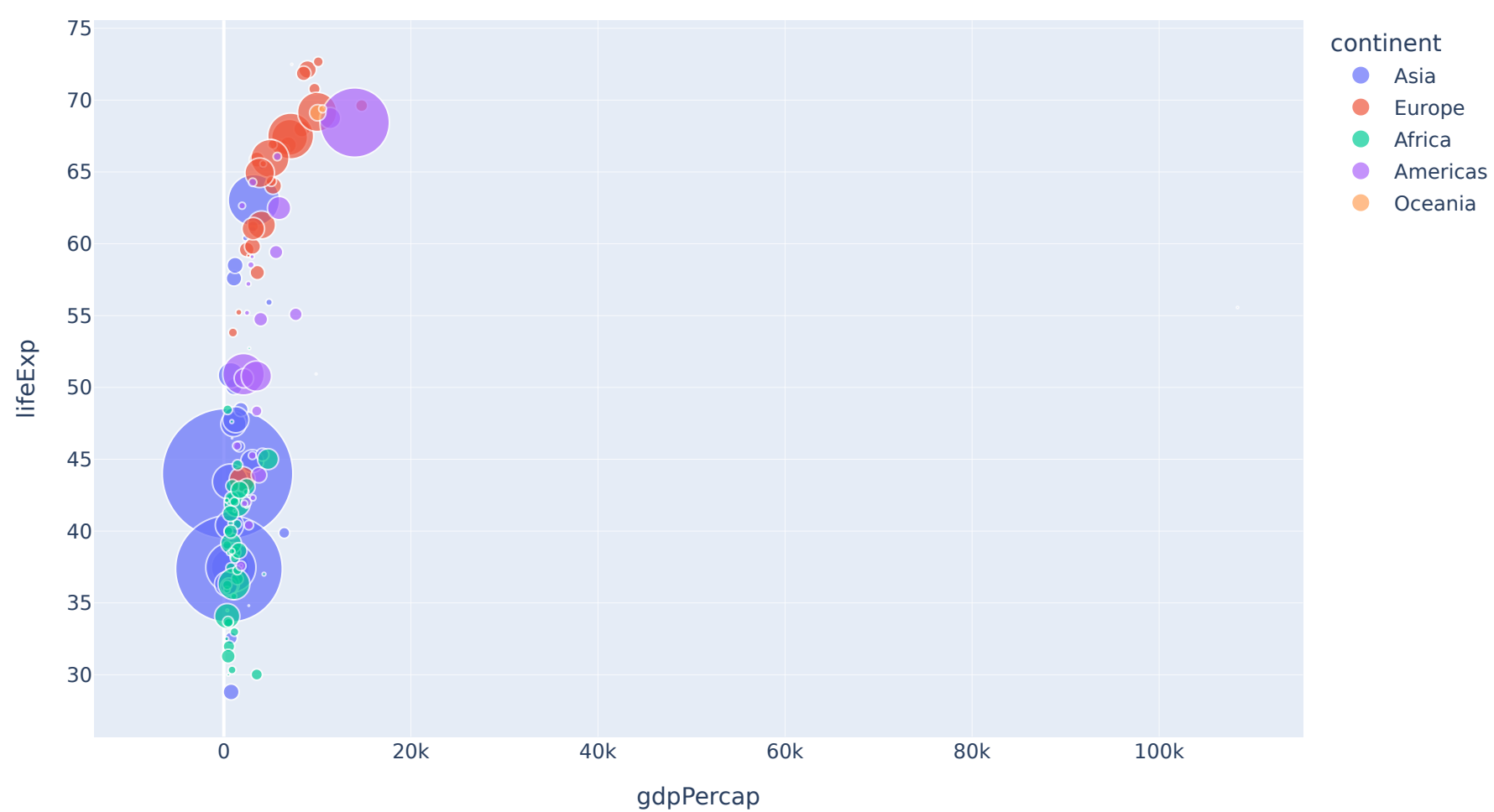
- **142力国 × 12時点（1952～2007年、5年おき）** の実データ

まず1年分だけ描いてみる

- `gap[gap["year"] == 1952]` で1952年の行だけに絞る（Pandasの条件抽出）
- あとは ここまでと同じ `x, y, size, color, hover_name`

Code

```
1 gap_1952 = gap[gap["year"] == 1952]
2
3 fig = px.scatter(
4     gap_1952,
5     x="gdpPercap",
6     y="lifeExp",
7     size="pop",
8     size_max=55,
9     color="continent",
10    hover_name="country",
11 )
12 fig.show()
```



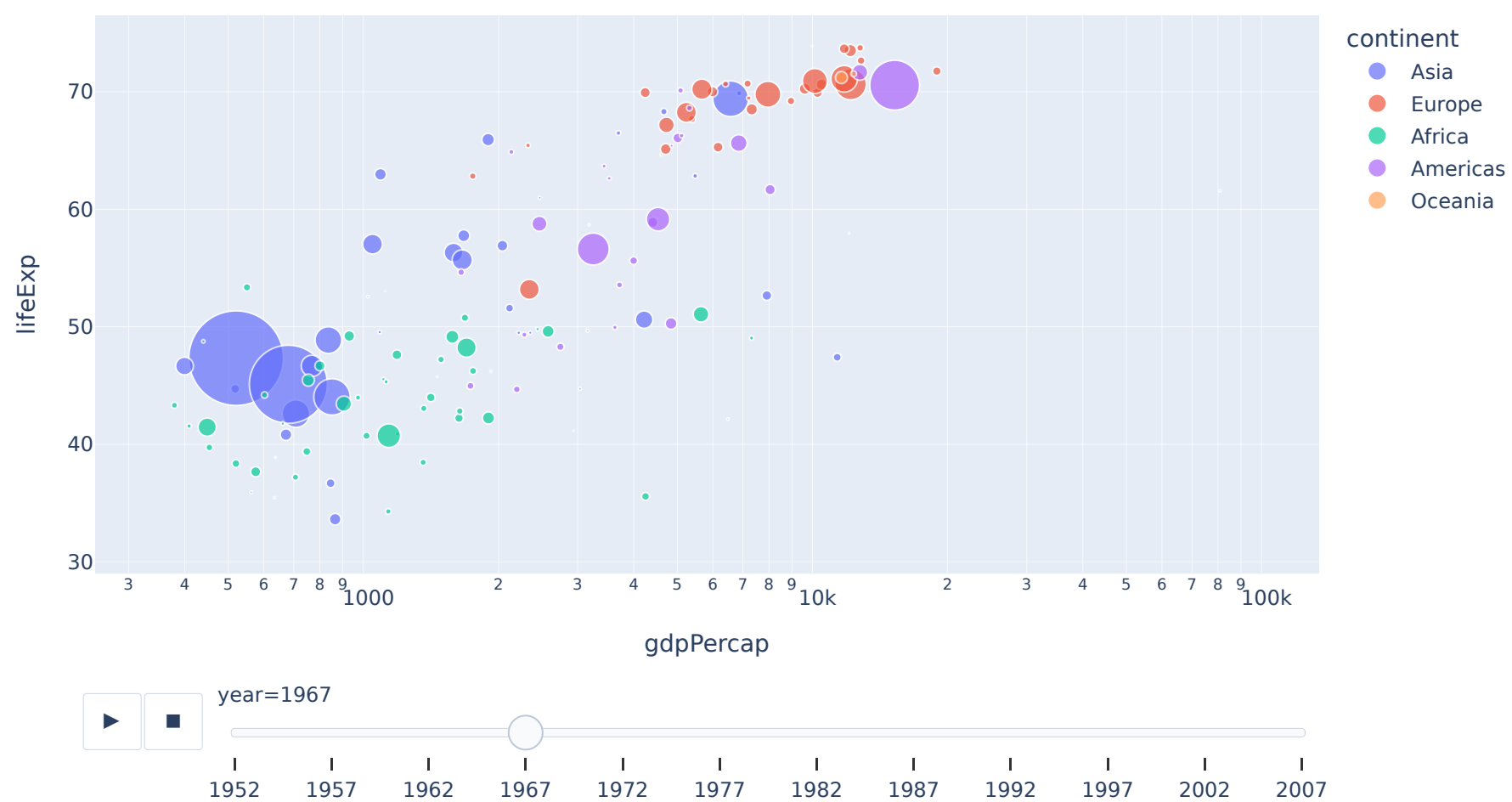
GDPは数百～10万ドル超まで桁違いに広がり、ほとんどの国が左端に固まって見える

動くバブルチャートにする

- さっきのコードから変えるのは **3か所** だけ：
 - 絞り込みをやめて **gap 全体** を渡す
 - **animation_frame="year"** — **year** の値ごとにコマを作って動かす
 - **log_x=True** — 横軸を対数軸（**log scale**）にして、左端の団子を解消

Code

```
1 fig = px.scatter(  
2     gap,  
3     x="gdpPercap",  
4     y="lifeExp",  
5     size="pop",  
6     size_max=55,  
7     color="continent",  
8     hover_name="country",  
9     log_x=True,  
10    animation_frame="year",  
11 )  
12 fig.show()
```



- ▶ **を押す** と55年分の世界が動く。気になるバブルにマウスを乗せると国名が出る



- さっきのコード を1か所ずつ変えて実行

- | | | |
|---|----------|--|
| 1 | 対数軸をやめる | <code>log_x=True</code> を消す |
| 2 | バブルを大きく | <code>size_max=100</code> |
| 3 | 大陸ごとに分ける | <code>facet_col="continent"</code> を足す |

💡 予想してから試す

- `log_x=True` を消すと、ほとんどの国はどこに固まる？（1952年の図がヒント）

33

まとめ

- Plotlyは **インタラクティブな可視化**：ズーム・ホバー・凡例クリック・回転
- 基本形は `px.グラフの種類(df, x="列名", y="列名")`
- 色分けは `color=`、点の大きさは `size=`、ホバーの見出しは `hover_name=`
- アニメーションは `animation_frame=`
- Pandasの集計結果は **DataFrame + 列名** にしてから渡す（`as_index=False` か `reset_index()`）
- `write_html()` でHTMLとして保存して、動くまま共有できる

34